

Mathematical Foundations of Generative AI

Stanford Frontier AI Learning System

Baseline: October 6, 2026. Print companion to the HTML course site:
crash course, cheatsheet, course map, prerequisites, notation and shapes, glossary.

Contents

Crash course	2
Cheatsheet	7
Course map	11
Prerequisites	13
Notation and shapes	19
Glossary	24

Crash course

Date: 2026-10-07. Baseline: October 6, 2026.

Ten units, about five minutes each. Every worked number below comes from the named lesson. The crash course points, the lessons prove. This is a provisional independent bridge built from the course lessons. It does not claim to reproduce the instructor's lectures. Source attribution for leaf concepts is PENDING (see `source_manifest.md`, `source_gaps.md`).

U01, Probabilistic generative modelling (lessons/u01)

Three objects, never one. The data law p_{data} is the unknown truth. The empirical law p_{hat} is counts over n : the dataset as a distribution. The model law q is the formula you fit. Score q against data with log-likelihood, never with vibes.

The running toy: eight tiny files, 4-bit patterns. 0000 appears 3 times, 1111 appears 2 times, 0101 appears 2 times, 1010 appears 1 time. So p_{hat} gives 0000 probability 0.375, 1111 and 0101 probability 0.25 each, 1010 probability 0.125.

Worked number (U01): entropy $H(p_{\text{hat}}) = 1.9056$ bits. Cross-entropy against the uniform law = 2.0 bits. So $KL(p_{\text{hat}} || \text{uniform}) = 2.0 - 1.9056 = 0.0944$ bits. The uniform law wastes 0.0944 bits per file on this toy.

Memory aid: mass sums to 1, density integrates to 1. Likelihood is a function of the parameters, not the data. Zero mass on real data is fatal: one impossible sample kills the whole likelihood.

Self-test: Q: Why is held-out likelihood the honest score? A: Training likelihood rewards memorization. Held-out data punishes it.

U02, Variational divergence minimization (lessons/u02)

Jensen's inequality is the engine of every bound in this course. For concave f , $f(E[X]) \geq E[f(X)]$. Log is concave, so log of an expectation bounds the expectation of the log. That one move creates the ELBO and every variational bound after it.

Worked number (U02): $E[X] = 0.55$, $f(E[X]) = 0.3025$, $E[f(X)] = 0.325$. Jensen gap = 0.0225. The bound sits 0.0225 below the truth, and the gap is exactly the price of the approximation.

Divergences have dual faces. The variational dual writes KL as a max over test functions T : $KL = \max_T E_p[T] - \log E_q[e^T]$. Worked number (U02): dual value 0.1838 nats = 0.2651 bits, matching the direct KL to all shown digits. The dual turns a ratio into an optimization.

Worked number (U02): Monte Carlo with $N = 10000$ gives estimate 1.001125 with predicted standard error 0.006124. Error shrinks like $1/\sqrt{N}$: 100x samples buys 10x precision. Gradients check out too: analytic -0.3873127314 versus finite-difference -0.3873127312.

Memory aid: every bound has a gap, and the gap has a name. Name it or you cannot close it.

Self-test: Q: When is a variational bound tight? A: When the approximating family contains the truth. Then the gap is zero.

U03, GAN foundations (lessons/u03)

Two networks, one game. The generator fakes. The discriminator scores real versus fake. The minimax objective: $\min_G \max_D E[\log D(x)] + E[\log(1 - D(G(z)))]$. At the optimum, the discriminator reports the density ratio $D^* = p_{\text{data}} / (p_{\text{data}} + p_g)$, and the game value equals $-\log 4 + 2 * \text{JSD}(p_{\text{data}} || p_g)$. Train the generator and you minimize the Jensen-Shannon divergence without ever writing a density.

Worked number (U03): at the toy setting the max_D value reads -1.3537, and $2 * \text{JS} - 2 = -1.3537$, so $\text{JS} = 0.3231$ bits. The identity holds at every generator position in the sweep, error $4.4e-16$. The math is exact, the training is not.

The original loss saturates. When D is perfect, the generator gradient dies. The non-saturating fix flips the generator to maximize $\log D(G(z))$ instead. Same fixed point, live gradients early. Mode collapse is the failure the loss cannot see: the generator covers one mode, the discriminator cannot punish missing modes, recall dies silently.

Memory aid: the discriminator is a ratio meter. Perfect accuracy means saturation, not convergence.

Self-test: Q: The discriminator hits 100 percent on real versus fake. Good news? A: No. The generator gets zero gradient and training stalls.

U04, Wasserstein and improved adversarial training (lessons/u04)

When two laws share no support, JS reads 1 bit and KL reads infinity, and neither moves when the generator shifts. The toy: $P_0 = \delta_0$, $P_\theta = \delta_\theta$, $\theta = 1$. $\text{JS} = 1$ bit, $\text{KL} = \text{infinity}$, both frozen. You need a ruler that sees the gap of 1.

The Wasserstein-1 distance is the cheapest transport cost: $W_1 = \min \text{over couplings of } E[|x - y|]$. On the toy, $W_1 = |\theta| = 1.0$ with gradient $dW_1/d\theta = 1.0$. Finite, nonzero, pointing the right way. The Kantorovich dual turns it into a max over 1-Lipschitz functions: $W_1 = \max_{\{f || f|_L \leq 1\}} E_p[f] - E_q[f]$. The critic replaces the discriminator, and the Lipschitz cap replaces the sigmoid.

Enforcement is the whole game. Weight clipping (C05) gives 0.01 on the critic toy: an underestimate that starves capacity. The gradient penalty enforces the constraint on interpolations instead: a heuristic with live gradients, not a proof. Training stabilizes because the critic signal stays linear in the gap.

Memory aid: JS asks "same or different". W_1 asks "how far". Only the second trains on disjoint laws.

Self-test: Q: Why does the WGAN critic drop the sigmoid? A: It estimates a distance, not a probability. The output must stay unbounded.

U05, Variational autoencoders (lessons/u05)

The marginal $p(x) = \sum_z p(x, z)$ is intractable, so Jensen (U02) builds the ELBO: $\log p(x) \geq E_q[\log p(x|z)] - \text{KL}(q(z|x) || p(z))$. Reconstruction minus rate. The encoder proposes q , the decoder scores, the KL prices the code.

Worked number (U05): on the toy with drawn $z = 0.824264$, reconstruction = -0.7342 nats (-1.0593 bits), $\text{KL} = 0.1766$ nats (0.2547 bits), one-sample ELBO = -0.9108 nats (-1.3140 bits). Both terms are costs. The sum is the bound, labeled as a bound.

The reparameterization trick moves the noise outside the parameters: $z = \mu + \sigma * \epsilon$. Gradients flow through μ and σ while ϵ stays frozen. The Gaussian KL has a closed form, so no sampling is needed for the rate term.

Posterior collapse is the silent killer: a powerful decoder ignores z , the KL reads ~ 0 , and the model is an autoregressive memorizer wearing a VAE costume. Beta scales the rate price along the rate-distortion frontier. Diagnose collapse by the KL per dim, not by the reconstructions.

Memory aid: rate buys distortion. Dead latents cost nothing and explain nothing.

Self-test: Q: KL reads 0.0003 per dim but samples look sharp. What happened? A: Collapse. The decoder memorized and ignores z .

U06, Discrete latent modelling and VQ-VAE (lessons/u06)

Continuous latents blur. Quantize them: the encoder emits z_e , the nearest codebook vector e_1 wins, the decoder sees only z_q . Three losses train the three parts. Reconstruction pulls the decoder. The codebook loss pulls the winner toward the encoder. The commitment loss pulls the encoder toward the winner.

Worked number (U06): $z_e = [0.9, 0.2]$, winner e_1 . Reconstruction $\|x - \text{decoder}(z_q)\|^2 = 0.05$. Codebook $\|sg[z_e] - e_1\|^2 = 0.05$. Commitment $0.25 * \|z_e - sg[e_1]\|^2 = 0.0125$. The stop-gradient $sg[.]$ routes each loss to its owner: no owner, no learning.

Argmin has no gradient, so the straight-through estimator copies the decoder gradient from z_q to z_e as if quantization were the identity. Worked number (U06): copied STE gradient $[0, -1]$ versus true gradient $[0, 0]$. The copy is biased by construction. It works when z_e sits close to z_q , which the commitment loss enforces.

Dead codes never win and never learn. EMA updates track usage counts N_k and sums m_k instead of backprop through the winners. The prior over codes is a separate autoregressive model: the VQ-VAE compresses, the prior generates.

Memory aid: three losses, three owners. The stop-gradient is the fence between them.

Self-test: Q: Half the codebook has zero usage after training. What happened? A: Dead codes. Winners took all. Restart them near live codes or raise the commitment weight.

U07, DDPM derivation and parameterizations (lessons/u07)

Destroy, then learn to un-destroy. The forward chain adds Gaussian noise on a schedule: $\beta_1 = 0.0001$ to $\beta_{100} = 0.02$, $T = 100$ steps. Direct noising jumps to any step: $x_t = \sqrt{\alpha_{\bar{t}}} x_0 + \sqrt{1 - \alpha_{\bar{t}}} \text{eps}$. The reverse step removes the predicted noise.

Worked number (U07): $x_0 = 2.0$, drawn noise $\text{eps} = 0.5$. At $t = 50$, $x_{50} = 1.99917538$. The network predicts $\text{eps}_{\text{hat}} = 0.4$ (true 0.5, error 0.1). Estimated clean $x_0_{\text{hat}} = 2.05354466$, reverse mean $\mu_{\text{rev}} = 2.00072225$. The 0.1 noise error moved the mean by 0.0007. The posterior coefficients shrink errors: small mistakes in noise space stay small in data space.

The ELBO collapses to the epsilon loss: predict the noise, nothing else. The parameterization choice (noise, clean x_0 , or score) changes what the network outputs but not the sampling rule. Boundary variance needs care: at t near 0 the posterior variance has two defensible choices and the wrong one shows in samples.

Memory aid: the schedule is the curriculum. Too fast and the signal dies early. Too slow and you waste steps on pure noise.

Self-test: Q: Why predict noise instead of x_0 directly? A: The noise target has unit scale at every t . x_0 prediction mixes scales across the chain and trains worse.

U08, Diffusion variants and implementation (lessons/u08)

DDPM needs all 100 steps. DDIM keeps the same marginals with a non-Markovian construction and strides the chain. Deterministic when $\eta = 0$, stochastic when $\eta = 1$. Fewer steps, same endpoints.

Worked number (U08): $\alpha_{\bar{50}} = 0.77718008$. One DDIM jump from 100 to 90: $x_{100} = 1.60480905$, predicted clean $x_0 = 2.13230847$, $x_{90} = 1.71491474$. Ten such jumps reach x_0 . Ten network calls instead of one hundred, same toy start.

The network must know what time it is. The U-Net takes the timestep embedding alongside the noisy input: same weights, time-conditioned behavior. Conditioning (class labels, text) steers the reverse process. Loss weighting rebalances the t values: uniform weighting wastes capacity on near-pure noise.

Memory aid: DDIM changes the path, not the destination. The marginals match by construction.

Self-test: Q: DDIM with 10 steps looks worse than DDPM with 100. What do you check first? A: The step schedule. Even spacing is rarely optimal. Spend steps where $\alpha_{\bar{}}$ moves fast.

U09, Score-based modelling and autoregressive models (lessons/u09)

The score is $\text{grad}_x \log p(x)$: it points uphill with no normalizer. Denoising score matching fits it by regression: perturb x with Gaussian noise, train the network to predict $(x - x_{\text{tilde}})/\sigma^2$. Low-density regions get signal because the noise puts them there.

Worked number (U09): two-mode mixture. $s(1.0) = 1.999926$, $s(1.5) = 0$, $s(-1.0) = -1.999926$. Zero at the modes and the valley, positive left of the right mode (pointing right, uphill). DSM target at $x_{\text{tilde}} = 1.7$ from clean $x = 1.5$ with noise var 0.04: target = -5.0. Langevin dynamics then walks uphill with calibrated noise: 5 steps from 0 read 0.0, 0.039759, 0.060272, 0.354732, 0.608576, 0.617063, drifting toward the right mode.

The SDE/ODE bridge unifies it: the forward SDE destroys, the reverse SDE (or probability-flow ODE) rebuilds, and the score is the only learned object. DDPM is one discretization of this picture.

Then the other family: autoregressive models. The chain rule factors the joint exactly: $p(x) = \prod p(x_i | x_{<i})$. Teacher forcing trains on true prefixes. Sampling feeds the model its own outputs (exposure bias). The causal mask is the arrow of time. The transformer implements it with attention at $O(L^2 d)$ per layer and a KV cache at sampling.

Memory aid: the score needs no partition function. The chain rule needs no approximation. Pick your poison honestly.

Self-test: Q: Why does the score blow up at distribution boundaries? A: $\log p$ dives to negative infinity there, so its gradient explodes. Likelihoods need a named noise floor.

U10, LLM inference, quantization, and alignment (lessons/u10)

Training is parallel, generation is serial. Temperature reshapes the draw without retraining: $\text{softmax}(z/T)$. Worked number (U10): $T = 0.5/1.0/2.0$ gives $p_{\text{max}} 0.8282/0.5745/0.4056$ and entropy 0.8754/1.6174/1.9017 bits. Cold sharpens, hot flattens. Top-p truncates the tail and renormalizes.

The KV cache is the whole economics. Attention at step $n+1$ needs K and V for positions $1..n$: cache them. Worked number (U10): $L = 12$, $h = 8$, $d_h = 64$, $n = 512$, fp16: cache = 12,582,912 bytes = 12.00 MiB, 24,576 bytes per new token. Prefill costs 3,221,225,472 FLOPs versus 6,291,456 per decode token: ratio

1. Decode is memory-bound, so shrinking the

model speeds it up linearly.

Worked number (U10): INT8 affine on a 4x2 toy: scale = $3.5/255 = 0.013725$, zero-point 87, max abs error 0.005882, mean abs error 0.004167. Bytes 32 $\rightarrow$ 8, a 4x win. Roofline: 7B fp16 at 14 GB and 2 TB/s gives 7.00 ms/token, 142.9 tok/s ceiling. INT4 at 3.5 GB gives 1.75 ms, 571.4 tok/s. All C04 numbers are authored arithmetic, not benchmarks.

Alignment, three stages. SFT teaches the format on demonstrations. Reward modelling scores pairs: $r_{\text{chosen}} = 1.2$ versus $r_{\text{rejected}} = 0.3$ on the toy. PPO climbs the reward with a KL leash to the reference. DPO skips the reward model and pushes the log-ratio margin directly, $\beta = 0.1$ on the toy, with the same KL regularizer keeping the policy near the reference. Reward hacking is the failure mode: the policy games the proxy, not the goal.

Memory aid: decode reuses the past, quantization shrinks the present, alignment steers the future. Three budgets, three knobs.

Self-test: Q: Doubling the context length doubles decode latency per token? A: Roughly yes for attention over the cache ($O(n)$ per step), but the cache bytes double too. Both budgets move together.

The whole course in ten lines

U01: three laws (data, empirical, model). Score with logs. Zero mass kills. U02: Jensen builds every bound. The gap always has a name. Monte Carlo error falls as $1/\sqrt{N}$. U03: the game minimizes JS at an optimum that never holds. Saturation kills gradients, not convergence. U04: JS and KL freeze on disjoint laws. $W1 =$

θ	moves with gradient 1.
----------	------------------------

U05: ELBO = reconstruction minus KL. The reparameterization trick routes gradients. Watch for collapse. U06: three losses, three owners. The straight-through copy is biased by design. Dead codes never learn. U07: destroy on a schedule, predict the noise. 0.1 noise error moves the mean 0.0007. U08: DDIM strides the chain, ten steps for one hundred. The network must know the time. U09: learn the uphill direction, walk it with noise. Or factor the joint exactly and pay $O(L^2)$. U10: cache the past (12 MiB per 512 tokens). Shrink the weights (4x for INT8). Steer with preferences (KL leash on).

Cheatsheet

Date: 2026-10-07. Baseline: October 6, 2026. One-stop dense reference for the 10 units. Numbers below are the lesson toys, not universal constants.

U01, Generative modelling basics

Object	Symbol	Meaning
Data law	p_{data}	Unknown truth
Empirical law	p_{hat}	Counts / n
Model law	q or p_{theta}	The fitted formula
Log-likelihood	$l = \sum \log q(x_i)$	Score in nats or bits
Entropy	$H = -E[\log p]$	Expected surprise
Cross-entropy	$H(p,q) = -E_p[\log q]$	Score of q under p
KL	$KL(p \parallel q)$	$KL(p \parallel q) = H(p,q) - H(p)$ Directed price of mismatch

Toy numbers (8 files): $H = 1.9056$ bits. $H(p_{\text{hat}}, \text{uniform}) = 2.0$ bits. $KL = 0.0944$ bits.

Rules: zero mass on real data kills the likelihood. Held-out likelihood is the honest score. Mass sums, density integrates.

U02, Variational bounds

Item	Formula
Jensen (concave f)	$f(E[X]) \geq E[f(X)]$
ELBO	$\log p(x) \geq E_q[\log p(x,z) - \log q(z)]$
Gap	$KL(q(z) \parallel p(z x))$
KL dual	$\max_T E_p[T] - \log E_q[e^{e^T}]$
MC error	$se = \sigma / \sqrt{N}$

Toy numbers: Jensen gap 0.0225. Dual 0.1838 nats = 0.2651 bits. MC $N=10000$: est 1.001125, se 0.006124. Gradient check: analytic -0.3873127314, finite-diff -0.3873127312.

Rules: every bound has a named gap. 100x samples buys 10x precision. Check gradients against finite differences before trusting them.

U03, GANs

Item	Formula
Minimax	$\min_G \max_D E[\log D(x)] + E[\log(1-D(G(z)))]$
Optimal D	$D^* = p_{\text{data}} / (p_{\text{data}} + p_g)$
At optimum	$\max_D V = -\log 4 + 2 \text{JSD}$
Non-saturating G	$\max_G E[\log D(G(z))]$

Toy numbers: max_D V = -1.3537, JS = 0.3231 bits, identity exact to 4.4e-16 across the sweep.

Rules: D at 100 percent means saturation, not convergence. Mode collapse is a recall failure the loss cannot see. Compare generators with recall, not with D's accuracy.

U04, Wasserstein GANs

Item	Formula
W1	$\min_{\text{coupling}} E[\ x - y\]$
Dual	$\max_{\{1\text{-Lipschitz } f\}} E_p[f] - E_q[f]$
Clipping	crude box on weights, biased
Gradient penalty	on interpolations, heuristic

Toy numbers (theta = 1): JS = 1 bit, KL = inf, both frozen. W1 = 1.0, dW1/dtheta = 1.0. Clipping critic toy: 0.01, an underestimate.

Rules: on disjoint laws only W1 trains. The Lipschitz cap is the whole game. Clipping starves capacity, the penalty keeps gradients alive.

U05, VAEs

Item	Formula
ELBO	$E_q[\log p(x z)] - KL(q(z x) \parallel p(z))$
Reparam	$z = \mu + \sigma * \text{eps}, \text{eps} \sim N(0,1)$
Gaussian KL	$0.5 (\mu^2 + \sigma^2 - 1 - \log \sigma^2)$
Beta-VAE	recon - beta * KL

Toy numbers: recon -0.7342 nats, KL 0.1766 nats, ELBO -0.9108 nats (-1.3140 bits).

Rules: KL near 0 per dim with sharp samples = collapse. Beta moves along rate-distortion. The KL term is load-bearing: drop it and q collapses to a point.

U06, VQ-VAE

Loss	Formula	Toy value
Reconstruction	$\ x - \text{dec}(z_q)\ ^2$	0.05
Codebook	$\ sg[z_e] - e\ ^2$	0.05
Commitment	$0.25 \ z_e - sg[e]\ ^2$	0.0125

STE: backward copies decoder grad from z_q to z_e. Toy: copied [0,-1], true [0,0]. Biased by design.

Rules: three losses, three owners, sg[.] is the fence. Dead codes (zero usage) never learn: restart near live codes. The prior over codes is a separate AR model.

U07, DDPM

Item	Formula
Forward	$x_t = \sqrt{ab_t} x_0 + \sqrt{1-ab_t} \epsilon$
Schedule	$\beta_{1e-4} \rightarrow 0.02, T = 100$
Reverse mean	$c_0 x_{0_hat} + c_t x_t$
x_{0_hat}	$(x_t - \sqrt{1-ab_t} \epsilon_{hat}) / \sqrt{ab_t}$
Loss	$E[\epsilon - \epsilon_{hat}(x_t, t)]^2$

Toy numbers: $x_{50} = 1.99917538$, $\epsilon_{hat} = 0.4$ (true 0.5). $x_{0_hat} = 2.05354466$, $\mu_{rev} = 2.00072225$. Error 0.1 in noise moves the mean 0.0007.

Rules: predict noise (unit scale every t), not x_0 . The schedule is the curriculum. Posterior coefficients shrink errors.

U08, Diffusion variants

Item	Notes
DDIM	non-Markovian, same marginals, strides
$\eta = 0$	deterministic. $\eta = 1$ stochastic
Timestep embed	network must know t
Conditioning	class/text steers reverse
Loss weighting	rebalance t, do not waste on pure noise

Toy numbers: $ab_{50} = 0.77718008$. Jump 100 $\rightarrow$ 90: 1.60480905 $\rightarrow$ 1.71491474. Ten jumps reach x_0 .

Rules: DDIM changes the path, not the destination. Spend steps where α_{bar} moves fast.

U09, Score models and AR models

Item	Formula
Score	$s(x) = \text{grad}_x \log p(x)$
DSM loss	$E[s(x_{tilde}) - (x - x_{tilde})/s]^2$
Langevin	$x \leftarrow x + (a/2) s(x) + \sqrt{a} z$
AR	$p(x) = \prod p(x_i x_{<i})$
Attention	$O(L^2 d)$ per layer

Toy numbers: $s(1.0) = 1.999926$, $s(1.5) = 0$. DSM target -5.0. Langevin: 0, 0.0398, 0.0603, 0.3547, 0.6086, 0.6171.

Rules: the score needs no normalizer. It explodes at boundaries (name a noise floor). AR likelihood is exact. Sampling is serial and suffers exposure bias.

U10, LLM inference and alignment

Knob	Effect (toy V=4)
T = 0.5 / 1.0 / 2.0	pmax 0.8282 / 0.5745 / 0.4056
Entropy	0.8754 / 1.6174 / 1.9017 bits
Top-p	truncate tail, renormalize

KV cache (L=12, h=8, d_h=64, n=512, fp16): 12,582,912 bytes = 12.00 MiB. 24,576 bytes per new token. Prefill 3,221,225,472 FLOPs vs decode 6,291,456 per token: ratio

1. Decode is memory-bound.

INT8 affine: scale $3.5/255 = 0.013725$, zero-point 87, max abs err 0.005882, mean 0.004167. Bytes 32 -> 8 (4x). 7B fp16: 7.00 ms/token, 142.9 tok/s ceiling. INT4: 1.75 ms, 571.4 tok/s. (Authored arithmetic, not benchmarks.)

Alignment: SFT (format) -> reward model (r_chosen 1.2 vs r_rejected 0.3) -> PPO with KL leash, or DPO direct (beta 0.1, log-ratios 0.4 / -0.3). Watch reward hacking: the proxy is not the goal.

Decision rules across the course

Question	Answer
Bound or exact?	AR exact. VAE bound. Flows exact. GAN game
Disjoint laws?	W1, never JS/KL
KL ~ 0, samples sharp?	Posterior collapse
D at 100%?	Saturation, training stalled
Fewer diffusion steps?	DDIM, check schedule
Decode too slow?	KV cache, then quantize (memory-bound)
Model misbehaves after RL?	Reward hacking. Check KL to reference
Dead codes?	Restart near live codes

Course map

Date: 2026-10-06. Playlist and repository references are title-level and path-level only (SRC-04, SRC-05). Transcript inspection remains open. The BSDA5002 week structure (SRC-02) is the binding unit map.

Unit to syllabus and playlist mapping (week level)

Unit	Parent topic	BSDA5002 weeks	Playlist weeks (titles)
math-genai-U01	Probabilistic generative modelling	Week 1	W1_L1 course outline. W1_L2 introduction and p
math-genai-U02	Variational divergence minimization	Week 2	W1L3 f-Divergence. W1L4 variational divergence
math-genai-U03	GAN foundations and applications	Week 3	W2_L6, W2_L7 GANs intro and formulation. W2
math-genai-U04	Wasserstein and improved adversarial training	Week 4	W4L11 Wasserstein GANs. W4L12 inversion with
math-genai-U05	Variational autoencoders	Week 5	W5L17 intro to latent variable models. W5L18 EL
math-genai-U06	Discrete latent modelling and VQ-VAE	Week 6	W6L24 beta-VAE. W6L25 VQ-VAE. W6T13 VQ-V
math-genai-U07	DDPM derivation and parameterizations	Week 7	W7L26 DDPMs. W7L27 DDPM formulation. W7
math-genai-U08	Diffusion variants and implementation	Week 8-9	W9L34 alternate interpretations of DDPMs. W9
math-genai-U09	Score-based models and autoregressive LMs	Week 9-10	W9L34-L35 score reading of DDPM (bridge). W
math-genai-U10	LLM inference, quantization, and alignment	Week 11-12	W11L47 overview of RL. W11L48 policy gradient

Syllabus week structure (SRC-02, official)

Week 1 Introduction to Probabilistic Deep Generative Modelling. Week 2 Generative Modelling via variational Divergence Minimization. Week 3 Generative Adversarial Networks: Part 1 (Introduction and Formulation). Week 4 Generative Adversarial Networks: Part 2 (WGANs and Applications). Week 5 Generative Modelling via Variational Auto Encoding. Week 6 Variational Auto Encoders: Improvisations and VQVAE. Week 7 Denoising Diffusion Probabilistic Models (DDPMs) - Formulation. Week 8 Diffusion Models: Multiple forms and Implementation. Week 9 Conditional Diffusion Models and Score-based models. Week 10 Auto-Regressive Models and Large Language Models Introduction. Week 11 LLMs: Models, Sampling, Inference and Quantization Methods. Week 12 LLMs Reinforcement Learning based Alignment Methods (PPO, DPO).

Prerequisite dependency order for RUN 2 onward

Build P03, P06, P07, P08 before U02 deepens (P05, P09, P18 for U02 come next). U03 needs P09, P11. U04 needs P04 added. U05 needs P18. U06 needs P04. U07 needs P05, P18. U08 needs P12. U09 needs P13, P14. U10 needs P15, P17. Shared modules live in `shared/prerequisites/` and are already built. This course adds only the local remediation it needs.

Figure and assessment status

- U01: figure f01_data_vs_model.png in visual_audit.md. Diagnostic diagnostic-01. Lesson exercises E01-E20 + ladders L01-L05, keys separate.
- U02-U10: figures and assessments planned. No artifacts yet.

Prerequisites

Date: 2026-10-06. The shared 24-module bridge (P01-P24) already exists at `~/workspace/stanford-frontier-ai/v2-pack/shared`. This file gives the course-level graph plus self-contained local remediation for the P03, P06, P07, and P08 essentials that U01 needs. Notation follows the shared modules (log base 2 in bits, $I(X, Y)$ with a comma).

Module index (from the prerequisite atlas)

P01 numeracy/algebra/notation. P02 Python and scientific software. P03 vectors/geometry/linear maps. P04 spectral and numerical linear algebra. P05 scalar and multivariable calculus. P06 probability. P07 statistical estimation and uncertainty. P08 information theory. P09 optimization and constrained problems. P10 ML foundations and evaluation. P11 neural networks and autodiff. P12 PyTorch, tensors, numerical stability. P13 language and sequence modelling. P14 transformer mechanics. P15 hardware and computer architecture. P16 distributed systems. P17 reinforcement learning. P18 Bayesian inference, latent variables, sampling. P19 retrieval. P20 tools, APIs, agent state. P21 security and privacy. P22 experimental method. P23 economics. P24 production ML.

Dependency chains into this course

- U01 needs P03 (needs P01), P06 (needs P01), P07 (needs P04, P06),

P08 (needs P06, P07).

- U02 needs P05 (needs P01, P03), P08, P09 (needs P04, P05),

P18 (needs P06, P07, P08, P09).

- U03 needs P08, P09, P11 (needs P02, P03, P05, P09, P10).

- U04 needs P04 (needs P03), P08, P09.

- U05 needs P08, P11, P18.

- U06 needs P04, P09, P11, P18.

- U07 needs P05, P06, P08, P18.

- U08 needs P09, P11, P12 (needs P02, P11), P18.

- U09 needs P05, P08, P13 (needs P06, P08, P10),

P14 (needs P11, P12, P13), P18.

- U10 needs P12, P14, P15 (needs P01, P02), P17 (needs P06, P09,

P10).

Local remediation: P03 essentials used by U01

R1, a file is a vector

A file with d numbers is a point in $\mathbb{R}^d$. A 4-bit pattern 0101 is the vector $[0, 1, 0, 1]$ in $\mathbb{R}^4$. Shape (4,). Concrete: $x = [0, 1, 0, 1]$. $x_2 = 1$ (index, not exponent).

R2, dot product

Multiply matching items, then add. Concrete: $[0, 1, 0, 1]$ dot $[1, 1, 1, 1] = 0 + 1 + 0 + 1 = 2$. Rule: the dot product counts shared 1s for binary vectors.

R3, vector mean

Add the vectors, divide by the count. Concrete: $[0, 0, 0, 0]$ and $[1, 1, 1, 1]$ have mean $[0.5, 0.5, 0.5, 0.5]$. Check: computed 2026-10-06, numpy 1.26.4, float64.

Local remediation: P06 essentials used by U01

R4, sample space and events

The sample space lists every possible outcome. An event is a subset. Concrete: sample space of one coin flip is {heads, tails}. The event "heads" has probability $1/2$ for a fair coin.

R5, probability mass function

A probability mass function assigns one number to each outcome. The numbers are in $[0, 1]$ and sum to 1. Concrete: fair coin: $P(\text{heads}) = 0.5$, $P(\text{tails}) = 0.5$. Sum = 1. Misconception: a probability is not a count. The count 3 out of 8 is data. The probability $3/8$ is the model of that data.

R6, expectation

The expectation is the probability-weighted average. Concrete: for a fair coin paying 1 for heads and 0 for tails, $E = 0.5 \cdot 1 + 0.5 \cdot 0 = 0.5$.

Local remediation: P07 essentials used by U01

R7, maximum likelihood on a coin

Pick the parameter that makes the observed data most probable. Concrete: 7 heads in 10 flips. Candidate $\theta = 0.7$ gives log-likelihood $7 \log_2(0.7) + 3 \log_2(0.3) = -8.81$ bits. Candidate $\theta = 0.5$ gives -10.0 bits. $\theta = 0.7$ wins. Computed 2026-10-06.

R8, small-sample wobble

An estimate from few samples wobbles. One head in one flip gives $\theta = 1$, which claims a sure thing from one observation. Rule: distrust estimates whose sample count is tiny.

Local remediation: P08 essentials used by U01

R9, logs and bits

$\log_2(x)$ asks: 2 to what power gives x . One bit is one binary choice. Concrete: $\log_2(8) = 3$, $\log_2(1/4) = -2$. Rule: log of a product is the sum of logs. This is why log-likelihoods add across independent items.

R10, logs of small numbers

$\log_2$ of a tiny probability is a large negative number. A model that assigns probability 2^{-30} to the data pays 30 bits for that item. Rule: probabilities multiply, so log-probabilities add, and a near-zero probability dominates the sum.

Local remediation: P05 essentials used by U02

R11, derivative as a slope

The derivative df/dx is the slope of f at x . Concrete: $f(x) = x^2$, $df/dx = 2x$. At $x = 3$ the slope is 6. Check: $(3.001^2 - 3^2)/0.001 = 6.001$, close to 6.

R12, partial derivative and gradient

Hold all but one variable fixed, take the slope in that variable. The gradient collects the partials. Concrete: $g(a, b) = a^2 + 3b$. $dg/da = 2a$, $dg/db = 3$. Gradient at $(1, 5)$ is $[2, 3]$.

R13, convex function

A function is convex when every chord between two graph points lies above the graph. Concrete: x^2 is convex. $\sqrt{x}$ is concave (chords below). $x^3 - 3x$ is neither. Rule: convexity decides the Jensen direction.

Local remediation: P09 essentials used by U02

R14, gradient descent

Step against the gradient to lower a function. Concrete: minimize $(x-3)^2$ from $x = 0$, step 0.1: $x \leftarrow x - 0.1 \cdot 2(x-3)$. First step: $x = 0.6$. It walks toward 3.

R15, supremum

The supremum is the least upper bound: no member exceeds it, and members approach it. Concrete: $\sup$ of $\{1 - 1/n\}$ is 1, never attained. The dual maximizes over functions. The $\sup$ may sit at a limit point.

Local remediation: P18 essentials used by U02

R16, Monte Carlo

Estimate an expectation by a sample mean. Concrete: 1000 draws from Bernoulli(0.7) with seed 0 give mean 0.678 (numpy 1.26.4, computed 2026-10-06). Error scale: $\sigma/\sqrt{N}$. Rule: predict the error bar before measuring.

R17, ELBO in one line

$\log p(x) \geq E_q[\log p(x,z) - \log q(z)]$. The gap is $KL(q(z) \parallel p(z|x))$. Concrete: the C03 toy gives bound -1.4490, gap 0.4490, sum -1.0 = $\log p(x)$. Computed 2026-10-06.

R18, importance identity

$E_p[g] = E_q[g * p/q]$ when supports allow. Concrete: with $r = [0.5, 1.75]$, $E_q[r] = 1.0$ exactly. Rule: ratios average to 1 under q . Use it as the first check of any ratio estimate.

Local remediation: P09 essentials used by U03

R19, min-max games and saddle points

$\min_x \max_y f(x, y)$: read inside out. For each x , y plays its best, giving $g(x) = \max_y f(x, y)$. Then minimize g over x . A saddle point (x^*, y^*) is a minimum in x and a maximum in y at the same point. Concrete: $f(x, y) = (x - y)^2 - y^2$... simpler: $f(x, y) = x*y$. For fixed x , max over y is unbounded unless y is bounded: restrict y to $[-1, 1]$. Then $g(x) = |x|$, minimized at $x^* = 0$ with $y^* = \text{sign}(x)$. The GAN game is $\min_G \max_D V$: D 's best response first, then G . Rule: the inner max must track the outer player, or the number means nothing.

Local remediation: P11 essentials used by U03

R20, a neuron

A neuron is an affine map plus a nonlinearity: $\text{out} = \text{sigma}(w \cdot x + b)$. w is a weight vector, b a bias scalar, sigma a fixed function (sigmoid, ReLU). Concrete: $w = [2]$, $b = -1$, $x = 1$, $\text{sigma} = \text{sigmoid}$: $\text{out} = \text{sigmoid}(1) = 0.7311$. Shape: x in $\mathbb{R}^d$, w in $\mathbb{R}^d$, out a scalar. A layer stacks neurons. A net is layers composed.

R21, backprop is the chain rule

To train, we need $d \text{ loss} / d w$ for every weight. The chain rule: $d \text{ loss} / d w = (d \text{ loss} / d \text{ out})(d \text{ out} / d w)$. Backprop applies it layer by layer, from the loss backward. It computes every gradient in one forward pass plus one backward pass: $O(\text{params})$, not $O(\text{params}^2)$. Concrete: $\text{loss} = (\text{out} - 1)^2$, $\text{out} = \text{sigmoid}(w x + b)$, $x = 1$, $w = 2$, $b = -1$. $d \text{ loss} / d \text{ out} = 2(\text{out} - 1) = -0.5379$. $d \text{ out} / d w = \text{sigmoid}'(1) * 1 = 0.1966$. Product: -0.1058 . Check by finite differences: $(\text{loss}(2.001) - \text{loss}(1.999))/0.002 = -0.1058$. Computed 2026-10-06.

R22, the training loop

Repeat: sample a batch, forward pass (compute the loss), backward pass (gradients), step against the gradient (R14). For GANs there are two loops interleaved: ascend on D, descend on G (U03-C02 pseudocode). The loop is the same. Only the sign and the player change.

Local remediation: P04 essentials used by U04

R23, Lipschitz maps and the spectral norm

A map f is L -Lipschitz when $|f(a) - f(b)| \leq L |a - b|$ for all a, b . L is the maximum slope. Concrete: $f(x) = 2x$ is 2-Lipschitz. $f(x) = -x$ is 1-Lipschitz. For an affine map $x \rightarrow Wx$, the Lipschitz constant is the largest singular value of W . Concrete: $W = \begin{bmatrix} 2 \\ -1 \end{bmatrix}$ has $sv = \sqrt{5} = 2.2361$, so $x \rightarrow Wx$ is 2.2361-Lipschitz. Composition multiplies constants: $L(f \text{ composed with } g) \leq L(f) L(g)$. ReLU is 1-Lipschitz. Check: computed 2026-10-06, numpy 1.26.4, float64.

R24, gradient norm as a local slope

For smooth f , $|\text{grad } f(x)|$ is the local slope at x . The Lipschitz constant is the max of $|\text{grad } f|$ over the whole space. Concrete: $f(x) = 2.5x$ has $\text{grad } 2.5$ everywhere, so it is 2.5-Lipschitz. Finite-difference check: $(f(x+h) - f(x-h)) / (2h)$ with $h = 1e-6$ gives 2.49999999996. Rule: a penalty on $|\text{grad } f|$ at samples controls the slope only where it is measured.

Diagnostic routing

diagnostics/diagnostic-01-genai-foundations.md probes R1-R10 and the U01 concepts. A score below 70 percent sends the learner back to the matching R-section before U02. R11-R18 are probed inside the U02 lesson exercises (E01-E03 assume R13, E14-E15 assume R16, E16-E17 assume R11-R12, E18-E19 assume R15). A learner failing those is sent to the matching R-section before the U02 ladders. R19-R22 are probed inside the U03 lesson exercises (E03 assumes R19, E09 assumes R21, E19 assumes R14+R22, E02 assumes R20). A learner failing those is sent to the matching R-section before the U03 ladders. R23-R24 are probed inside the U04 lesson exercises (E07-E08 assume R23, E11-E12 assume R24, E05 assumes R15+R19). A learner failing those is sent to the matching R-section before the U04 ladders. R25-R26 are probed inside the U05 lesson exercises (E03 assumes R25, E02 assumes R26) and reused by U07 (E08 assumes R25). A learner failing those is sent to the matching R-section before the U05 ladders. R27 is probed inside the U08 lesson exercises (E09-E10 assume R27 tensor shapes, E11 assumes R27). A learner failing those is sent to R27 before the U08 ladders. R28-R29 are probed inside the U09 lesson exercises (E07-E08 assume R28, E10-E12 assume R29). A learner failing those is sent to the matching R-section before the U09 ladders. R30-R31 are probed inside the U10 lesson exercises (E04-E05 and E07-E08 assume R30, E10-E13 assume R31). A learner failing those is sent to the matching R-section before the U10 ladders.

R25, Gaussian facts for P18 (used by U05, U07)

$N(\mu, \sigma^2)$ has density $(2\pi\sigma^2)^{-0.5} \exp(-(x - \mu)^2 / (2\sigma^2))$. $E[x] = \mu$, $\text{Var}(x) = \sigma^2$. Linear transforms stay Gaussian: $a x + b \sim N(a\mu + b, a^2\sigma^2)$. Sums of independent Gaussians are Gaussian with summed means and variances. The KL between $N(\mu, s^2)$ and $N(0, 1)$ is $0.5(\mu^2 + s^2 - 1 - \log s^2)$ nats. Conditioning two Gaussians gives a Gaussian (used in the DDPM posterior). Check: computed 2026-10-06, numpy 1.26.4, float64.

R26, the ELBO five-line proof (U02 to U05 bridge)

Line 1: $\log p(x) = \log \int p(x | z) p(z) dz$. Line 2: multiply and divide by $q(z | x)$ inside. Line 3: $\log p(x) = \log E_q[p(x | z) p(z) / q(z | x)]$. Line 4: Jensen (log concave): $\geq E_q[\log p(x | z) + \log p(z) - \log q(z | x)]$. Line 5: group the last two terms as $-KL(q || p)$. Result: $\log p(x) \geq E_q[\log p(x | z)] - KL(q || p)$. The gap is $KL(q || p(z | x))$. Check: the toy gap 1.3040 nats, computed 2026-10-06.

R27, tensor shapes for P12 (used by U08)

A tensor has a shape: a list of axis lengths. The toy U-Net uses (C, H, W): channels, height, width. A conv 1→8 k3 on 1x8x8 gives 8x8x8: channels change, space stays (stride 1, padding 1). A matmul (m, k) @ (k, n) gives (m, n): the inner axes must match. Broadcasting: (4,) + (3, 4) works (the 4 aligns), (3,) + (4,) fails. no_grad/detach: marks a tensor as not needing gradients (used for targets like the drawn eps, and for the frozen x_T in the determinism check). dtype bytes: fp32 4, fp16 2, int8 1. Check: (2, 3) @ (3, 2) → (2, 2), computed 2026-10-06, numpy 1.26.4, float64.

R28, sequence modelling basics for P13 (used by U09)

A token is an integer id for a text piece, the vocab lists all of them (V = 4 in the toy: a, b, c, d). A sequence is a list of ids. The AR factorization: $p(x_1..x_n) = \prod p(x_i | x_{<i})$. Teacher forcing: train each factor on the true prefix. Perplexity: $2^{\{NLL/n\}}$, the effective branching factor, in [1, V]. Embeddings: each id becomes a vector (the E matrix). Check: the bigram NLL 1.73697 bits and perplexity 1.82574, computed 2026-10-06.

R29, attention basics for P14 (used by U09)

Queries, keys, values: three projections of the input. Scores: $QK^T/\sqrt{d}$. Weights: softmax over keys per query (rows sum to 1). Output: the weighted average of V. The causal mask zeroes the future (upper triangle) so position i sees only 0..i. Scale $1/\sqrt{d}$ keeps the softmax out of saturation. Check: the 3x3 toy weights rows [1,0,0], [0.3302,0.6698,0], [0.2483,0.2483,0.5035], computed 2026-10-06.

R30, hardware basics for P15 (used by U10)

Bytes: params times bytes-per-element (fp32 4, fp16 2, int8 1, int4 0.5). Bandwidth: bytes per second the memory delivers. Latency: time per op. Decode is memory-bound: each token reads all weights once, so tok/s ≤ bandwidth / model bytes (the roofline). Prefill is compute-bound: $O(n^2)$ FLOPs dominate. Always roofline both limits, quoting FLOP/s for a memory-bound workload is true and irrelevant. Check: 7B fp16 = 14e9 bytes, at 2e12 B/s the ceiling is 142.9 tok/s. Authored arithmetic, computed 2026-10-06.

R31, RL basics for P17 (used by U10)

An MDP: states, actions, rewards, transitions. A policy $\pi(a|s)$: the action distribution. The return: discounted sum of rewards. The value: expected return from a state. The policy gradient trick: $\text{grad } E[\text{return}] = E[\text{grad } \log \pi(a|s) * \text{advantage}]$: actions better than average get more probable. PPO: clip the policy ratio to [1-e, 1+e] so steps stay small. The Bradley-Terry model: $P(A \text{ beats } B) = \text{sigmoid}(r_A - r_B)$. For an AR-LM: state = tokens so far, action = next token, reward = preference score. Check: the bandit toy: two actions, rewards 1 and 0, $\pi = \text{softmax}([0, 0]) = [0.5, 0.5]$. $d/\text{dlogits } \log \pi(a1) = [0.5, -0.5]$, with advantage 1 the update raises $\pi(a1)$. Computed 2026-10-06.

Notation and shapes

Date: 2026-10-06. Follows the shared symbol contract in `shared/prerequisites/notation_and_shapes.md`. Symbols are introduced before first use in the U01 lesson.

U01 symbols

Symbol	Meaning	First defined	
x	one file as a vector in $\mathbb{R}^d$ (a 4-bit pattern in the toy)	R1	
d	dimension of x . Toy: $d = 4$	R1	
x_i	i -th item of x (index). $x_2 = 1$ for $x = [0, 1, 0, 1]$	R1	
D	dataset, a list of n items	C01	
n	dataset size. Toy: $n = 8$	C01	
p_{data}	true (unknown) distribution that produced D	C01	
p_{hat}	empirical distribution from D (counts / n)	C01	
q, q_{theta}	model distribution, with parameters θ	C01	
θ	model parameter (a number the model can change)	C01	
$p(x), q(x)$	probability the distribution assigns to item x	R5	
sum_x	sum over every possible x	R5	
$E_p[f]$	expectation of f under p . $\text{sum}_x p(x) f(x)$	R6	
$L(\theta)$	likelihood: product of q_{theta} over the n items of D	C03	
$l(\theta)$	log-likelihood: sum of $\log q_{\text{theta}}$ over the n items of D	C03	
$\log$	log base 2 everywhere in this course, unless stated	R9	
bits	unit of information. One bit is one binary choice	R9	
support	set of x with $p(x) > 0$	C04	
z	latent variable (hidden, never observed)	C05	
$p(x, z)$	joint probability of x and z	C06	
$p(x z)$	conditional probability of x given z	C06	
$p(z x)$	posterior probability of z given observed x	C06	
$H(p)$	entropy of p : $E_p[-\log p(x)]$, in bits	C07	
$H(p, q)$	cross-entropy: $E_p[-\log q(x)]$, in bits	C08	
$D_{\text{KL}}(p \parallel q)$	KL divergence: $E_p[-\log q(x)] - E_p[-\log p(x)]$	C08	
$I(X, Y)$	mutual information of X and Y (comma form, no semicolon)	glossary	
$p(z)$	prior distribution over the latent variable	C06	

U02 symbols

Symbol	Meaning	First defined	
λ	convex weight in $[0, 1]$	C01	
f (generator)	convex function with $f(1) = 0$, selects the divergence	C05	
$D_f(p \parallel q)$	$\parallel$	q)	f-divergence: $E_q[f(p/q)]$, in bits C05
$r(x)$	density ratio $p(x)/q(x)$, unit-free	C04	
$d(x)$	classifier $P(x \text{ came from } p)$. $r = d/(1-d)$	C04	
$T(x)$	witness function in the dual	C06	
$f^*(s)$	convex conjugate: $\sup_t (s t - f(t))$	C06	
$\sup$	supremum: least upper bound	R18	
ϕ	variational family parameter vector	C02	
q_ϕ	family member at ϕ	C02	
ϵ	smoothing mass for support repair	C07	
N	Monte Carlo sample count	C08	
σ	true std of the integrand under the sampling distribution	C08	
SE	standard error: $\sigma / \sqrt{N}$	C08	
$J(a, b)$	dual objective over witness parameters, in nats	C09	
$\text{Bernoulli}(p)$	coin with $P(\text{heads}) = p$	lesson toy	

Log base 2 in bits for divergences. Natural log (nats) inside conjugate computations, converted at the end.

U03 symbols

Symbol	Meaning	First defined	
G, θ_G	generator network and its weights	C01	
D, θ_D	discriminator network and its weights	C01	
z	noise input to G , usually $N(0, I)$	C01	
$V(D, G)$	game value: $E_{p_D}[\log_2 D] + E_{p_G}[\log_2 (1-D(G(z)))]$, bits	C01	
$D^*_G(x)$	optimal discriminator: $p_{\text{data}}(x)/(p_{\text{data}}(x)+p_g(x))$	C03	
p_g	generator's distribution (pushforward of noise)	C01	
m	midpoint density: $(p_{\text{data}} + p_g)/2$	C04	
$JS(p \parallel q)$	$\parallel$	q)	Jensen-Shannon divergence, bits, max 1 C04
t	discriminator logit: $D = \text{sigmoid}(t)$	C05	
k	discriminator steps per generator step	C02	
y	conditioning label (conditional GAN)	C08	
E	encoder network: $x \rightarrow z$ (BiGAN)	C10	
ϵ	clip floor keeping D in $[\epsilon, 1-\epsilon]$	C07	

Shape conventions for this course

- One file x is a column vector in $\mathbb{R}^d$. Toy: $d = 4$, shape (4,).
- A dataset of n files is the list D , not one big matrix, at U01.
- Probabilities are scalars in $[0, 1]$. Log-probabilities are negative numbers or zero. Information is in bits.
- A distribution over K outcomes is a list of K numbers that sum to 1 (a probability vector).

U04 symbols

Symbol	Meaning	First defined	
$W_1(p, q)$	Wasserstein-1 distance: min coupling cost, data units	C01	
γ	coupling: joint with marginals p and q	C01	
$c(x, y)$	ground cost, here	$x - y$	C02
f	critic: real-valued witness in the dual	C03	
$L(f)$	Lipschitz constant of f	C03	
$E_p[f] - E_q[f]$	dual gap, in data units	C03	
$sv(W)$	largest singular value of W (spectral norm)	C04	
c	weight clip bound. Toy: $c = 0.01$	C05	
λ	gradient-penalty weight. Toy: $\lambda = 10$	C06	
$\hat{x}$	interpolated point: $\epsilon x + (1-\epsilon) y$	C06	
n_{critic}	critic steps per generator step. Authored: 5	C08	
z^*	inverted latent code: argmin_z		$G(z) - x$ 2 C09
E	encoder in latent regression / BiGAN (U03 reuse)	C09	
d	domain label bit (domain adversarial)	C09	

U05 symbols

Symbol	Meaning	First defined	
$q(z x)$	encoder: approximate posterior over the latent code	C01	
$p(x z)$	decoder: likelihood of x given the code	C01	
$\mu(x), \sigma^2(x)$	encoder outputs: posterior mean and variance	C01	
$\log \text{var}$	$\log \sigma^2$, the encoder's unconstrained output	C03	
ELBO	$E_q[\log p(x z)] - \text{KL}(q$		
ϵ	frozen noise, $N(0, 1)$, in reparameterization	C04	
β	KL weight in the beta-VAE objective	C08	
γ_{ik}	GMM responsibility of component k for point i	SB	
$\text{KL}(q p)$			0.5 (mu

U06 symbols

Symbol	Meaning	First defined		
e_k	codebook entry k , in $\mathbb{R}^D$	C01		
K	codebook size. Toy: $K = 4$	C01		
z_e	encoder continuous output	C02		
z_q	quantized code: $e_{\{k^*\}}$	C02		
k^*	argmin_k		$z_e - e_k$	2 C02
$\text{sg}[\cdot]$	stop-gradient operator	C03		
N_k, M_k	EMA count and embedding sum for code k	C05		
γ	EMA decay. Toy: 0.99	C05		
R	rate: $\log_2 K$ bits per code	C09		
D	distortion: mean		$z_e - z_q$	2 (rate/distortion only) C09

U07 symbols

Symbol	Meaning	First defined
β_t	per-step noise variance. Toy: $1e-4$ to 0.02	C02
α_t	$1 - \beta_t$	C01
α_{bar}_t	cumulative product of α_s , $s = 1..t$	C03
SNR_t	$\alpha_{\text{bar}}_t / (1 - \alpha_{\text{bar}}_t)$	C03
x_t	noisy state at step t	C01
β_{tilde}_t	posterior variance: $(1 - \alpha_{\text{bar}}_{t-1}) / (1 - \alpha_{\text{bar}}_t) \beta_t$	C05
c_0, c_t	posterior mean coefficients on x_0 and x_t	C05
ϵ_{hat}	network noise prediction	C06
x_0_{hat}	predicted x_0 from ϵ_{hat}	C06
μ_{rev}	reverse mean from x_0_{hat}	C06
σ_t^2	reverse variance choice (boundary)	C11
$L_T, L_{\{t-1\}}, L_0$	ELBO terms: prior, middle KLs, decoder	C07

U08 symbols

Symbol	Meaning	First defined
τ	DDIM subsequence of timesteps, length S	C01
η	DDIM stochasticity knob in $[0, 1]$	C01
$\sigma_t(\eta)$	per-jump noise scale	C01
x_0_{hat}	implied clean prediction from ϵ_{hat}	C01
$\text{PE}(t)$	sinusoidal timestep embedding vector	C05
$w(t)$	per-timestep ELBO loss weight	C06
s_{hat}	score estimate: $-\epsilon_{\text{hat}} / \sqrt{1 - \alpha_t}$	C07
ϵ_u, ϵ_c	unconditional / conditional noise prediction	C09
g	classifier-free guidance scale	C09
S	number of DDIM evals	C03

U09 symbols

Symbol	Meaning	First defined
$s(x)$	score: $d/dx \log p(x)$	C01
x_{tilde}	noise-perturbed sample	C02
a	Langevin step size	C03
P (bigram)	4x4 next-token matrix, rows sum to 1	C05
ppl	perplexity: $2^{\{NLL/n\}}$	C05
A (attention)	masked softmax weights, (n, n)	C07
Q, K, V	query, key, value projections	C08
T (temp)	sampling temperature (U10 reuse)	C10
h, L	attention heads, layers (budget)	C11

U10 symbols

Symbol	Meaning	First defined
T	decoding temperature	C01
p (top-p)	nucleus truncation threshold	C01
bpe	bytes per element (fp16: 2)	C02
s, zp	quantization scale and zero-point	C03
r_c, r_r	Bradley-Terry rewards, chosen/rejected	C06
ρ	PPO policy ratio π/π_{old}	C07
e	PPO clip range. Toy: 0.2	C07
β	DPO/KL strength. Toy: 0.1	C08
π_{ref}	reference (SFT) policy	C08

Glossary

Date: 2026-10-06. One-line definitions, module pointers for depth.

- Data distribution (p_{data}): the unknown law that produced the dataset. U01-C01.
- Model distribution (q): the formula we fit to stand in for p_{data} . U01-C01.
- Empirical distribution (p_{hat}): counts divided by n . The dataset as a distribution. U01-C01.
- Density: probability per point for continuous x . U01-C02.
- Sample: one observed draw from a distribution. U01-C02.
- Likelihood (L): product of model probabilities over the dataset. U01-C03.
- Log-likelihood (l): sum of log model probabilities. U01-C03.
- Support: outcomes with positive probability. U01-C04.
- Latent variable (z): an unobserved variable that explains structure in x . U01-C05.
- Marginalization: sum (or integral) over z to get $p(x)$ from $p(x, z)$. U01-C06.
- Joint ($p(x, z)$): probability of x and z together. U01-C06.
- Conditional ($p(x | z)$): probability of x given z . U01-C06.
- Posterior ($p(z | x)$): probability of z after seeing x . U01-C06.
- Entropy (H): expected surprise of p , in bits. U01-C07.
- Surprisal: $-\log p(x)$. Bits needed to encode one draw. U01-C07.
- Cross-entropy ($H(p, q)$): expected surprise of p measured with q 's probabilities, in bits. U01-C08.
- KL divergence (D_{KL}): cross-entropy minus entropy. Extra bits paid for using q instead of p . U01-C09.
- Divergence: any score of difference between two distributions. U01-C10.

- Estimator: a rule that turns data into a parameter value.

U01-C11.

- Maximum likelihood (MLE): the estimator that maximizes

log-likelihood. U01-C11.

- Bias: systematic error of an estimator. U01-C11.
- Held-out data: data the model did not train on. U01-C12.
- Evaluation: scoring a model on held-out data or on samples.

U01-C12.

- Overfit: fit the training data at the price of held-out performance. U01-C12.
- Mutual information ($I(X, Y)$): reduction in uncertainty about X from knowing Y , in bits. RUN 2.
- Parameter (θ): a knob inside the model family. U01-C01.

U02 terms

Date: 2026-10-06.

- Convex function: every chord lies above the graph. U02-C01.
- Jensen inequality: $f(E[X]) \leq E[f(X)]$ for convex f . U02-C01.
- Jensen gap: $E[f(X)] - f(E[X])$, non-negative for convex f .

U02-C01.

- Variational family (Q): restricted parameterized set of candidate distributions. U02-C02.
- Lower bound: tractable $E_q[\log p(x, z) - \log q(z)]$ below $\log p(x)$. U02-C03.
- Density ratio (r): $p(x)/q(x)$, unit-free. U02-C04.
- f -divergence (D_f): $E_q[f(p/q)]$ for convex f , $f(1) = 0$.

U02-C05.

- Generator (f): the convex knob selecting the divergence.

U02-C05.

- Convex conjugate (f^*): $\sup_t (s^T t - f(t))$. U02-C06.
- Witness (T): the function maximized in the dual. U02-C06.
- Primal: minimize the divergence over distributions. U02-C06.
- Dual: maximize over functions, needs samples only. U02-C06.
- Support mismatch: $p > 0$ where $q = 0$. U02-C07.
- Smoothing: epsilon mass on every outcome. U02-C07.
- Monte Carlo: sample-mean estimate of an expectation. U02-C08.
- Standard error: $\sigma/\sqrt{N}$. U02-C08.
- Approximation gap: restricted-class sup below the full sup.

U02-C10.

- Estimation gap: empirical dual versus population dual.

U02-C10.

- Optimization gap: found maximizer versus best in class.

U02-C10.

- Consistency: convergence to the truth as N and the class grow under four assumptions. U02-C11.

U03 terms

Date: 2026-10-06.

- Generator (G): network mapping noise z to samples. U03-C01.
- Discriminator (D): network mapping x to $(0, 1)$, its bet that x is real. U03-C01.
- Game value (V): $E_{p_d}[\log_2 D] + E_z[\log_2(1-D(G(z)))]$, in bits. U03-C01.
- Minimax: min over G of max over D of V . U03-C02.
- Nash equilibrium: $p_g = p_{\text{data}}$, $D = 1/2$, $V = -2$ bits. U03-C02.
- Optimal discriminator (D^*): $p_{\text{data}}/(p_{\text{data}}+p_g)$, the Bayes ratio. U03-C03.

- JS divergence: $(KL(p_d||m) + KL(p_g||m))/2$, m the midpoint.

Max 1 bit. U03-C04.

- Saturation: minimax G gradient -D dying where D is

confident. U03-C05.

- Non-saturating loss: $E_z[-\log D(G(z))]$, gradient

$-(1-D)$. U03-C05.

- Mode collapse: generator assigns ~ 0 mass to a data

mode. U03-C06.

- Log-boundary explosion: $1/(D \ln 2)$ gradient spikes as

$D \rightarrow 0$. U03-C07.

- Conditional GAN: $G(z, y)$, $D(x, y)$, one game per class.

U03-C08.

- DCGAN: strided convs, batchnorm, ReLU/LeakyReLU, tanh

output. U03-C09.

- BiGAN: pair discriminator $D(x, z)$ matching the G-joint

and the E-joint. U03-C10.

- Update balance: k D-steps per G-step. U03-C11.

- Coverage: generator mass near a data mode. U03-C12.

- Coupling (γ): a joint distribution with marginals p and

q . A transport plan. U04-C01.

- Transport cost: $\sum \gamma(i, j) |x_i - y_j|$, the work of a

plan. U04-C01.

- Wasserstein-1 (W_1): the cheapest transport cost over all

couplings. In data units. U04-C02.

- Ground cost: $c(x, y)$, the per-unit move price. Here $|x - y|$.

U04-C02.

- Kantorovich-Rubinstein dual: $W_1 = \max$ over 1-Lipschitz f of

$E_p[f] - E_q[f]$. U04-C03.

- Critic (f): the real-valued witness in the W dual. Not a

classifier. U04-C04.

- Lipschitz constant (L): the maximum slope of f . U04-C04.
- Spectral norm: the largest singular value of a weight matrix.

The affine map's Lipschitz constant. U04-C04.

- Weight clipping: forcing every weight into $[-c, c]$ after each critic step. U04-C05.

- Gradient penalty: $\lambda E[(\|\text{grad } f(\hat{x})\| - 1)^2]$ on real-fake interpolation points. U04-C06.

- Interpolated point ($\hat{x}$): $\epsilon x + (1-\epsilon) y$, ϵ in $[0,1]$.

U04-C06.

- Calibration ratio: critic gap divided by exact $W1$ on a reference pair. U04-C07.

- Critic frequency (n_{critic}): critic steps per generator step.

U04-C08.

- GAN inversion: $z^* = \text{argmin } \|G(z) - x\|^2$, the latent code for a target. U04-C09.

- Latent regression: a learned encoder E trained on $(G(z), z)$ pairs. U04-C09.

- Domain adversarial training: task loss minus λ times domain loss, with the domain gradient reversed. U04-C09.

- Gradient reversal: flipping the sign of the domain-classifier gradient into the feature extractor. U04-C09.

- Notebook evidence: opened, executed, hash-recorded artifact content. File names are not evidence. U04-C10.

- Fair comparison: same architectures, same seeds, fixed flop budget, reported calibration and coverage. U04-C12.

U05 terms (2026-10-06)

- Prior $p(z)$: the model's belief about the latent code before seeing x . U05-C01.
- Encoder $q(z | x)$: the approximate posterior. Outputs μ and $\log\text{var}$ per x . U05-C01.
- Decoder $p(x | z)$: the likelihood of x given a code. U05-C01.
- ELBO: $E_q[\log p(x | z)] - \text{KL}(q || p)$. A lower bound on $\log p(x)$. U05-C02.
- Reconstruction term: $E_q[\log p(x | z)]$. Rewards codes that decode well. U05-C02.
- Reparameterization: $z = \mu + \sigma \epsilon$ with $\epsilon \sim N(0, 1)$ fixed. Makes sampling differentiable. U05-C04.
- Score-function gradient: $E[f(z) \text{grad} \log q(z)]$. Valid for any q , high variance. U05-C04.
- Posterior collapse: $q(z | x) = p(z)$ for all x . The model ignores the latent code. U05-C07.
- Beta-VAE: $E_q[\log p(x | z)] - \beta \text{KL}$. One knob trades reconstruction against latent use. U05-C08.
- Latent traversal: sweep z , decode each point. The cheapest collapse diagnostic. U05-C09.
- Amortized inference: one trained encoder answers every new x in one forward pass. U05-C11.
- Responsibility: γ_{ik} , the soft component assignment in a GMM E-step. U05 source block.
- EM: alternate soft assignments (E-step) and weighted updates (M-step). Monotone likelihood. U05 source block.

U06 terms (2026-10-06)

- Codebook: the table of K learned vectors in $\mathbb{R}^D$. The discrete alphabet. U06-C01.
- Vector quantization: replace z_e with the nearest codebook entry. U06-C02.
- Stop-gradient (`sg[. ]`): freezes its argument in backprop. U06-C03.
- Codebook loss: $\|sg[z_e] - e_k\|^2$. Moves the entry toward the encoding. U06-C03.
- Commitment loss: $\beta \|z_e - sg[e_k]\|^2$. Moves the encoding toward the entry. U06-C03.
- Straight-through estimator: forward z_q , backward copy the gradient to z_e . Biased but useful. U06-C04.
- EMA update: the codebook tracks running means of assigned encodings with horizon $1/(1 - \gamma)$. U06-C05.
- Dead code: an entry no z_e ever selects. Usage 0. U06-C06.
- Effective K : the number of codes with nonzero usage. U06-C06.
- Index prior: $p(k)$ over code indices. Stage 2 of the VQ-VAE generative story. U06-C07.
- Rate: $\log_2 K$ bits per code. U06-C09.
- Distortion: $\text{mean } \|z_e - z_q\|^2$. U06-C09.
- Controlled ablation: vary exactly one factor, hold seed/data/architecture fixed. U06-C12.

U07 terms (2026-10-06)

- Forward process: the fixed Markov chain $q(x_t | x_{t-1}) = N(\sqrt{1 - \beta_t} x_{t-1}, \beta_t)$. U07-C01.
- Noise schedule: the list $\beta_1.. \beta_T$. U07-C02.
- α_{bar}_t : the cumulative product of $(1 - \beta_s)$. The signal fraction surviving at t . U07-C03.
- SNR_t : $\alpha_{\text{bar}}_t / (1 - \alpha_{\text{bar}}_t)$. U07-C03.
- Direct noising: $x_t = \sqrt{\alpha_{\text{bar}}_t} x_0 + \sqrt{1 - \alpha_{\text{bar}}_t} \epsilon$. One jump, no chain. U07-C04.
- Posterior coefficient: the blend weights on x_0 and x_t in $q(x_{t-1} | x_t, x_0)$. U07-C05.
- $\tilde{\beta}_t$: $(1 - \alpha_{\text{bar}}_{t-1}) / (1 - \alpha_{\text{bar}}_t) * \beta_t$. The posterior variance. U07-C05.
- Reverse Gaussian: $p(x_{t-1} | x_t) = N(\mu_{\text{rev}}, \sigma_t^2)$ with x_0 replaced by x_0_{hat} . U07-C06.
- Epsilon prediction: the network outputs ϵ_{hat} . x_0_{hat} follows by algebra. U07-C08.
- Simplified loss: $\| \epsilon - \epsilon_{\text{hat}}(x_t, t) \|^2$ over uniform t . U07-C10.
- Boundary variance: the ELBO-underdetermined choice of σ_t^2 (β_t , $\tilde{\beta}_t$, or 0). U07-C11.
- Sampling loop: from $x_T \sim N(0, 1)$, iterate the reverse Gaussian T times. U07-C12.

U08 terms (2026-10-06)

- DDIM: a non-Markovian diffusion construction with the same marginals as DDPM, allowing big jumps. U08-C01.
- Marginal contract: $q(x_t | x_0)$ fixed, process free. The condition that lets one net serve many samplers. U08-C01.
- Eta: the stochasticity knob: 0 is deterministic (ODE), 1 is stochastic (SDE-like). U08-C02.
- Step schedule (τ): the subsequence of timesteps the sampler visits. U08-C03.
- Timestep embedding: sinusoidal vector form of t . U08-C05.
- Loss weighting: per-timestep multiplier, ELBO vs simplified. U08-C06.
- Tweedie estimate: $\hat{x}_0$ from the score estimate. U08-C07.
- Classifier-free guidance: $\epsilon_u + g(\epsilon_c - \epsilon_u)$. U08-C09.
- Hit-rate: toy quality proxy $P(|\hat{x}_0 - \text{target}| < 0.5)$. U08-C10.
- Check battery: the six-assert sampler contract test. U08-C12.

U09 terms (2026-10-06)

- Score: $\text{grad log } p(x)$, the uphill vector field. U09-C01.
- Denoising score matching: regression of the score on noise-perturbed samples. U09-C02.
- Langevin dynamics: score drift plus noise for MCMC sampling. U09-C03.
- Probability-flow ODE: the deterministic twin of the diffusion SDE with the same marginals. U09-C04.
- Chain rule: $p(x_1..x_n) = \prod p(x_i | x_{<i})$. U09-C05.
- Teacher forcing: training each factor on the true prefix. U09-C06.
- Exposure bias: the train/test prefix mismatch and its cost.

U09-C06.

- Causal mask: lower-triangular attention mask. U09-C07.
- Perplexity: $2^{\{NLL/n\}}$, the effective branching factor. U09-C09.
- Context budget: the $O(n^2)$ attention memory price of n .

U09-C11.

U10 terms (2026-10-06)

- Temperature: the softmax sharpening/flattening knob. U10-C01.
- Top-p (nucleus): truncation to the smallest set with mass $\geq p$.

U10-C01.

- KV cache: stored keys and values making decode $O(n)$ per token.

U10-C02.

- Prefill: the one $O(n^2)$ pass over the prompt. U10-C02.
- Quantization: fewer bits per weight via scale and zero-point.

U10-C03.

- Roofline: $\text{tok/s} \leq \text{bandwidth} / \text{model bytes (decode)}$. U10-C04.
- SFT: cross-entropy on demonstration responses. U10-C05.
- Bradley-Terry model: $P(\text{chosen} > \text{rejected}) = \text{sigmoid}(r_c - r_r)$.

U10-C06.

- PPO: clipped online policy optimization. U10-C07.
- DPO: offline preference loss on policy log-ratios. U10-C08.
- KL regularization: the price of drifting from π_{ref} . U10-C09.
- Reward hacking: the policy exploiting the proxy. U10-C10.
- Win-rate: mean BT probability over held-out pairs. U10-C11.